

LangGraph Runtime Postgres

开源情况调研报告

官方 Postgres Runtime 架构分析与 postgres_py 项目定位

2026-06-15

目录

1. 调研背景与目的
2. 官方开源声明
3. 官方架构分析
4. 开源范围界定
5. PyPI 包发布情况
6. 自研 postgres_py 定位
7. 技术选型建议
8. 参考资料

1. 调研背景与目的

在开发 langgraph_runtime_postgres_py 项目的过程中，我们需要明确以下问题：

- 官方是否已经开源了 LangGraph Runtime Postgres?
- 如果开源了，开源的范围和程度如何？
- 官方实现与我们的自研方案有何差异？
- 自研方案是否仍有价值？

本报告通过查阅官方文档、博客、GitHub 仓库以及分析本地安装的代码包，对以上问题进行解答。

2. 官方开源声明

LangChain 于 2025年1月28日 在官方博客宣布开源 LangGraph Runtime for Postgres：

- 博客标题："Announcing LangGraph Runtime for Postgres"
- 原文表述："We're excited to announce the open source release of LangGraph Runtime for Postgres."
- 定位：生产级、自托管、开源的运行时方案

LangGraph 平台层级

层级	类型	说明
Postgres Runtime	开源 (自托管)	生产级、自管理运行时，支持 PostgreSQL 持久化
Local Runtime	免费	仅限本地开发使用
Enterprise	商业	企业级支持，需联系销售

官方文档明确将 Postgres Runtime 定义为 "Open source, self-hosted"（开源、自托管）。

3. 官方架构分析

通过分析本地安装的 langgraph_api 0.10.0 代码，我们发现了官方的架构实现方式。

3.1 核心发现：Go + Python 双进程架构

官方 Postgres Runtime 并非纯 Python 实现，而是采用 Go + Python 双进程架构：

- Python 层：langgraph-api 包，提供 HTTP API 接口
- Go 层：独立的持久化服务，通过 gRPC 与 Python 层通信

3.2 代码证据：feature_flags.py

在 langgraph_api/feature_flags.py 中定义了 edition 检测逻辑：

```
_RUNTIME_EDITION = os.getenv("LANGGRAPH_RUNTIME_EDITION", "inmem")
IS_POSTGRES_BACKEND = _RUNTIME_EDITION == "postgres"
IS_POSTGRES_OR_GRPC_BACKEND = IS_POSTGRES_BACKEND
```

3.3 多态分发模式

API 路由根据 edition 选择不同的 ops 实现：

```
# langgraph_api/api/threads.py
from langgraph_api.feature_flags import IS_POSTGRES_OR_GRPC_BACKEND

if IS_POSTGRES_OR_GRPC_BACKEND:
    from langgraph_api.grpc.ops import Threads # gRPC client (■■ Go ■■)
else:
    from langgraph_runtime.ops import Threads # Python ■■
```

3.4 架构对比图解

架构	Python 进程	Go 进程	数据流向
官方 postgres	langgraph-api (HTTP)	持久化服务 (gRPC)	Python → gRPC → Go → PostgreSQL
官方 inmem	langgraph-api	无	Python → 内存字典
自研 postgres_py	langgraph-api	无	Python → psycopg3 → PostgreSQL

4. 开源范围界定

官方宣称的"开源"存在范围限制：

4.1 已开源部分

- langgraph-api: Python API 服务层，提供 REST API 和 WebSocket 接口
- langgraph-runtime-inmem: 纯 Python 的内存运行时，用于开发测试
- langgraph SDK: 核心图编排库
- checkpoint-postgres: PostgreSQL checkpoint saver (但仅限 checkpoint 层)

4.2 未开源部分

- Go 持久化服务: 官方 Docker 镜像中包含的二进制文件，源码未公开
- gRPC 协议定义: 部分 proto 文件未公开
- 企业级特性: 高级监控、集群支持等

4.3 开源程度总结

组件	开源状态	说明
Python API 层	□ 完全开源	langgraph-api 包, pip 可安装
Go 持久化服务	□ 未开源	仅提供 Docker 镜像中的二进制
gRPC 通信层	□□ 部分开源	Python 客户端开源, Go 服务端闭源
PostgreSQL Schema	□ 开源	可通过迁移文件获取

关键结论：官方 Postgres Runtime 的核心持久化逻辑（Go 服务）未开源，用户只能通过 Docker 镜像使用闭源二进制。

5. PyPI 包发布情况

我们查询了 PyPI 官方仓库：

5.1 已发布的包

包名	版本	说明
langgraph-api	0.10.0	Python API 服务，提供 HTTP/gRPC 接口
langgraph-runtime-inmem	0.30.0	内存运行时，用于开发测试
langgraph-checkpoint-postgres	2.0.x	PostgreSQL checkpoint saver

5.2 未发布的包

- langgraph-runtime-postgres：PyPI 上不存在此包
- 官方通过 Docker 镜像分发，而非 pip install

5.3 安装方式

官方推荐的安装方式：

```
# Docker Compose
docker run -d langchain/langgraph-api:latest \
-e LANGGRAPH_RUNTIME_EDITION=postgres \
-e DATABASE_URI=postgresql://... \
...
```

6. 自研 postgres_py 定位

基于以上调研，我们的 langgraph_runtime_postgres_py 项目具有独特价值。

6.1 方案对比

特性	官方 Postgres Runtime	自研 postgres_py
架构	Go + Python 双进程	纯 Python 单进程
核心代码	Go 层闭源	完全开源
依赖	Docker + Go 二进制	仅需 PostgreSQL + Redis
调试	需调试 Go + Python	纯 Python，易调试
可定制性	受限于闭源层	完全可控
部署	必须 Docker	支持 pip install
Edition	postgres	postgres_py

6.2 自研方案优势

- 完全开源：无需依赖闭源二进制，代码完全透明

- 轻量部署：无需 Docker，可直接 pip install 使用
- 易于调试：纯 Python 代码，IDE 断点调试无障碍
- 高度可控：可根据需求修改核心逻辑
- 学习价值：深入理解 LangGraph 运行时原理

6.3 自研方案劣势

- 需要自行维护 ops.py 接口与官方同步
- 缺少官方企业级特性支持
- 需要自行实现 TTL sweep 等后台任务

7. 技术选型建议

根据不同场景，给出以下建议：

7.1 选择官方 Postgres Runtime 的场景

- 企业生产环境，需要官方支持
- 不需要定制核心逻辑
- 接受使用 Docker 部署
- 不需要了解底层实现细节

7.2 选择自研 postgres_py 的场景

- 需要完全掌控代码，不接受闭源依赖
- 需要深度定制或二次开发
- 需要轻量级部署，不想依赖 Docker
- 用于学习研究 LangGraph 运行时原理
- 对代码透明度有严格要求的项目

7.3 总结

决策因素	官方方案	自研方案
生产稳定性	☐ 官方维护	☐ 自行负责
代码透明度	☐ 部分闭源	☐ 完全开源
部署复杂度	☐ 需要 Docker	☐ pip install
定制灵活性	☐ 受限	☐ 完全自由
技术支持	☐ 官方支持	☐ 自行解决

8. 参考资料

- 官方博客：<https://blog.langchain.dev/announcing-langgraph-runtime-for-postgres/>

- 平台概述: https://langchain-ai.github.io/langgraph/concepts/langgraph_platform/
- <https://langchain-ai.github.io/langgraph/how-tos/deploy/postgres-runtime/> Postgres Runtime
- GitHub 主仓库: <https://github.com/langchain-ai/langgraph>
- PyPI - langgraph-api: <https://pypi.org/project/langgraph-api/>
- PyPI - langgraph-runtime-inmem: <https://pypi.org/project/langgraph-runtime-inmem/>

— 报告结束 —